

1 源代码

Listing 1: main.cpp

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <math.h>
4 #include <time.h>
5 #include <stddef.h> // for size_t
6 #include <omp.h>
7 #include <chrono>
8 using Clock = std::chrono::high_resolution_clock;
9
10 /* 用固定种子生成确定性随机数 */
11 void generate_matrices(float *A, float *B, size_t NN)
12 {
13     srand(42); // 固定种子, 保证每次一致
14     for (size_t i = 0; i < NN; ++i)
15     {
16         A[i] = (float)rand() / (float)RAND_MAX * 2.0f -
17             1.0f;
18         B[i] = (float)rand() / (float)RAND_MAX * 2.0f -
19             1.0f;
20     }
21 }
22
23 void matmul_serial_C_TrC(const float *A, const float *B,
24     float *C, int N)
25 {
26     // this is output jump on mat C
27     // ci(j) = aik * bk(j)
28     long long NN = (long long)N * N;
```

```
26
27 // 清零结果矩阵
28 for (long long i = 0; i < NN; i++)
29     C[i] = 0.0f;
30
31 // 三重循环 i-k-j: jcache 友好型访问
32 for (int i = 0; i < N; i++)
33 {
34     for (int k = 0; k < N; k++)
35     {
36         float aik = A[(long long)i * N + k];
37         const float *b_row = &B[(long long)k * N];
38         float *c_row = &C[(long long)i * N];
39         for (int j = 0; j < N; j++)
40         {
41             c_row[j] += aik * b_row[j];
42         }
43     }
44 }
45
46 void matmul_serial_C_TrN(const float *A, const float *B,
47     float *C, int N)
48 {
49     // this is a plain mat mul
50     // cij = ai(k) * b(k)j
51     long long NN = (long long)N * N;
52
53     // 清零结果矩阵
54     for (long long i = 0; i < NN; i++)
55         C[i] = 0.0f;
```

```
56 // 三重循环 i-k-: jcache 友好型访问
57 for (int i = 0; i < N; i++)
58 {
59     for (int j = 0; j < N; j++)
60     {
61         const float *ai = &A[(long long)i * N];
62         for (int k = 0; k < N; k++)
63         {
64             C[i * N + j] += ai[k] * B[k * N + j];
65         }
66     }
67 }
68 }
69 void matmul_serial_C_TrB(const float *A, const float *B,
70 float *C, int N)
71 {
72     // this is a trans on B
73     // cij = ai(k) * bj(k)
74     long long NN = (long long)N * N;
75
76     // 清零结果矩阵
77     for (long long i = 0; i < NN; i++)
78         C[i] = 0.0f;
79
80     // 三重循环 i-k-: jcache 友好型访问
81     for (int i = 0; i < N; i++)
82     {
83         for (int j = 0; j < N; j++)
84         {
85             const float *ai = &A[(long long)i * N];
86             const float *bj = &B[(long long)j * N];
```

```

86         for (int k = 0; k < N; k++)
87         {
88             C[i * N + j] += ai[k] * bj[k];
89         }
90     }
91 }
92 }
93
94 void matmul_omp_C_TrC(const float *A, const float *B,
95     float *C, int N)
96 {
97     // this is output jump on mat C
98     // ci(j) = aik * bk(j)
99     long long NN = (long long)N * N;
100
101     #pragma omp parallel for schedule(static)
102     for (long long i = 0; i < NN; i++)
103         C[i] = 0.0f;
104
105     #pragma omp parallel for schedule(static)
106     for (int i = 0; i < N; i++)
107     {
108         for (int k = 0; k < N; k++)
109         {
110             float aik = A[(long long)i * N + k];
111             const float *b_row = &B[(long long)k * N];
112             float *c_row = &C[(long long)i * N];
113             for (int j = 0; j < N; j++)
114             {
115                 c_row[j] += aik * b_row[j];
116             }
117         }
118     }
119 }

```

```

116         }
117     }
118 }
119 void matmul_omp_C_TrN(const float *A, const float *B,
120                      float *C, int N)
121 {
122     // this is a plain mat mul
123     // cij = ai(k) * b(k)j
124     long long NN = (long long)N * N;
125
126     // 清零结果矩阵
127     #pragma omp parallel for schedule(static)
128     for (long long i = 0; i < NN; i++)
129         C[i] = 0.0f;
130
131     // 三重循环 i-k-j: jcache 友好型访问
132     #pragma omp parallel for schedule(static)
133     for (int i = 0; i < N; i++)
134     {
135         for (int j = 0; j < N; j++)
136         {
137             const float *ai = &A[(long long)i * N];
138             for (int k = 0; k < N; k++)
139             {
140                 C[i * N + j] += ai[k] * B[k * N + j];
141             }
142         }
143     }
144 void matmul_omp_C_TrB(const float *A, const float *B,
145                      float *C, int N)

```

```
145 {
146     // this is a trans on B
147     // cij = ai(k) * bj(k)
148     long long NN = (long long)N * N;
149
150     // 清零结果矩阵
151     #pragma omp parallel for schedule(static)
152     for (long long i = 0; i < NN; i++)
153         C[i] = 0.0f;
154
155     // 三重循环 i-k-j: jcache 友好型访问
156     #pragma omp parallel for schedule(static)
157     for (int i = 0; i < N; i++)
158     {
159         for (int j = 0; j < N; j++)
160         {
161             const float *ai = &A[(long long)i * N];
162             const float *bj = &A[(long long)j * N];
163             for (int k = 0; k < N; k++)
164             {
165                 C[i * N + j] += ai[k] * bj[k];
166             }
167         }
168     }
169 }
170 int main()
171 {
172     const int N = 1024; // 矩阵阶数
173     const long long NN = 1LL * N * N;
174     printf("N = %d\n", N);
175 }
```

```
176 // 生成确定性数据（便于复现）
177 float *A = (float *)malloc(sizeof(float) * NN);
178 float *B = (float *)malloc(sizeof(float) * NN);
179 float *C1 = (float *)malloc(sizeof(float) * NN);
180 float *C2 = (float *)malloc(sizeof(float) * NN);
181 if (!A || !B || !C1 || !C2)
182 {
183     printf("内存分配失败\n");
184     return 1;
185 }
186
187 generate_matrices(A, B, NN);
188 freopen("out.txt", "w", stdout);
189 {
190     printf("\循环顺序测试nikj\n");
191     // 串行
192     auto t0 = Clock::now();
193     matmul_serial_C_TrC(A, B, C1, N);
194     auto t1 = Clock::now();
195     double t_serial = std::chrono::duration<double>(
196         t1 - t0).count();
197
198     // 并行（可用 OMP_NUM_THREADS 控制线程数）
199     auto t2 = Clock::now();
200     matmul_omp_C_TrC(A, B, C2, N);
201     auto t3 = Clock::now();
202     double t_omp = std::chrono::duration<double>(t3
203         - t2).count();
204
205     // 结果校验（相对误差）
206     double max_rel_err = 0.0;
```

```

205     for (long long i = 0; i < NN; ++i)
206     {
207         double a = C1[i], b = C2[i];
208         double denom = std::max(1e-6, std::abs(a));
209         max_rel_err = std::max(max_rel_err, std::abs
            (a - b) / denom);
210     }
211
212     // 统计: 乘加算作 2 次浮点运算
213     const double flops = 2.0 * N * 1.0 * N * N;
214     double gflops_serial = flops / t_serial / 1e9;
215     double gflops_omp = flops / t_omp / 1e9;
216
217     int threads = 1;
218 #pragma omp parallel
219     {
220 #pragma omp master
221         threads = omp_get_num_threads();
222     }
223
224     printf("Threads: %d\n", threads);
225     printf("Serial    : %.3f s,  %.2f GFLOPS\n",
        t_serial, gflops_serial);
226     printf("OpenMP    : %.3f s,  %.2f GFLOPS\n",
        t_omp, gflops_omp);
227     printf("Speedup   : %.2fx\n", t_serial / t_omp);
228     printf("Max rel err: %.3e\n", max_rel_err);
229 }
230 {
231     printf("\nnijk 循环顺序测试\n");
232     // 串行

```



```

233     auto t0 = Clock::now();
234     matmul_serial_C_TrN(A, B, C1, N);
235     auto t1 = Clock::now();
236     double t_serial = std::chrono::duration<double>(
        t1 - t0).count();
237
238     // 并行 (可用 OMP_NUM_THREADS 控制线程数)
239     auto t2 = Clock::now();
240     matmul_omp_C_TrN(A, B, C2, N);
241     auto t3 = Clock::now();
242     double t_omp = std::chrono::duration<double>(t3
        - t2).count();
243
244     // 结果校验 (相对误差)
245     double max_rel_err = 0.0;
246     for (long long i = 0; i < NN; ++i)
247     {
248         double a = C1[i], b = C2[i];
249         double denom = std::max(1e-6, std::abs(a));
250         max_rel_err = std::max(max_rel_err, std::abs
            (a - b) / denom);
251     }
252
253     // 统计: 乘加算作 2 次浮点运算
254     const double flops = 2.0 * N * 1.0 * N * N;
255     double gflops_serial = flops / t_serial / 1e9;
256     double gflops_omp = flops / t_omp / 1e9;
257
258     int threads = 1;
259     #pragma omp parallel
260     {

```

```

261 #pragma omp master
262     threads = omp_get_num_threads();
263 }
264
265     printf("Threads: %d\n", threads);
266     printf("Serial    : %.3f s,  %.2f GFLOPS\n",
267           t_serial, gflops_serial);
267     printf("OpenMP    : %.3f s,  %.2f GFLOPS\n",
268           t_omp, gflops_omp);
268     printf("Speedup   : %.2fx\n", t_serial / t_omp);
269     printf("Max rel err: %.3e\n", max_rel_err);
270 }
271 {
272     printf("\nijk 转置测试B\n");
273     // 串行
274     auto t0 = Clock::now();
275     matmul_serial_C_TrB(A, B, C1, N);
276     auto t1 = Clock::now();
277     double t_serial = std::chrono::duration<double>(
278         t1 - t0).count();
279
280     // 并行 (可用 OMP_NUM_THREADS 控制线程数)
281     auto t2 = Clock::now();
282     matmul_omp_C_TrB(A, B, C2, N);
283     auto t3 = Clock::now();
284     double t_omp = std::chrono::duration<double>(t3
285         - t2).count();
286
287     // 结果校验 (相对误差)
288     double max_rel_err = 0.0;
289     for (long long i = 0; i < NN; ++i)

```

```
288     {
289         double a = C1[i], b = C2[i];
290         double denom = std::max(1e-6, std::abs(a));
291         max_rel_err = std::max(max_rel_err, std::abs
            (a - b) / denom);
292     }
293
294     // 统计: 乘加算作 2 次浮点运算
295     const double flops = 2.0 * N * 1.0 * N * N;
296     double gflops_serial = flops / t_serial / 1e9;
297     double gflops_omp = flops / t_omp / 1e9;
298
299     int threads = 1;
300     #pragma omp parallel
301     {
302         #pragma omp master
303             threads = omp_get_num_threads();
304     }
305
306     printf("Threads: %d\n", threads);
307     printf("Serial    : %.3f s,  %.2f GFLOPS\n",
            t_serial, gflops_serial);
308     printf("OpenMP    : %.3f s,  %.2f GFLOPS\n",
            t_omp, gflops_omp);
309     printf("Speedup   : %.2fx\n", t_serial / t_omp);
310     printf("Max rel err: %.3e\n", max_rel_err);
311 }
312
313 free(A);
314 free(B);
315 free(C1);
```

```
316     free(C2);  
317     return 0;  
318 }
```

2 测试结果

2.1 Debug

ikj循环顺序测试

Threads: 20

Serial : 1.256 s, 1.71 GFLOPS

OpenMP : 0.137 s, 15.73 GFLOPS

Speedup : 9.20x

Max rel err: 0.000e+00

ijk 循环顺序测试

Threads: 20

Serial : 3.370 s, 0.64 GFLOPS

OpenMP : 0.360 s, 5.97 GFLOPS

Speedup : 9.37x

Max rel err: 0.000e+00

ijk 转置B测试

Threads: 20

Serial : 2.001 s, 1.07 GFLOPS

OpenMP : 0.220 s, 9.76 GFLOPS

Speedup : 9.10x

Max rel err: 0.000e+00

2.2 Release

ikj循环顺序测试

Threads: 20

Serial : 0.095 s, 22.52 GFLOPS

OpenMP : 0.072 s, 29.84 GFLOPS

Speedup : 1.33x

Max rel err: 0.000e+00

ijk 循环顺序测试

Threads: 20

Serial : 2.411 s, 0.89 GFLOPS

OpenMP : 0.295 s, 7.27 GFLOPS

Speedup : 8.17x

Max rel err: 0.000e+00

ijk 转置B测试

Threads: 20

Serial : 0.510 s, 4.21 GFLOPS

OpenMP : 0.205 s, 10.49 GFLOPS

Speedup : 2.49x

Max rel err: 0.000e+00